SIH 2026 · PROBLEM STATEMENT 26249 · MINISTRY OF DEFENCE (DSSC)

VayuRakshak.

AeroTwin-AI — Predictive Maintenance & Fleet Availability Platform

A sovereign, air-gapped maintenance decision system that fuses aircraft telemetry, physics-based degradation models and gradient-boosted Remaining Useful Life prediction to keep combat aircraft mission-ready.

React 19 + TypeScript | FastAPI + scikit-learn | Physics + ML | 3D Digital Twin | e-MMS / IMMOLS interfaces

Technical dossier for presentation: software stack, ML model, physics engine, data layer, validation results and deployment posture.

AT A GLANCE

10 AIRCRAFT	**6** AIRBASES
4 FIGHTER TYPES	**8** CONSOLE MODULES

01 Problem
02 Solution
03 Architecture
04 Software stack
05 ML model
06 Physics & twin
07 Modules & data
08 Validation
09 Prototype
10 Deployment
11 Summary

01 · THE OPERATIONAL PROBLEM

Availability is lost between the aircraft and the depot.

Military air fleets lose mission-capable hours to disconnected data, reactive maintenance and parts that arrive after the aircraft is already grounded. The platform targets the full closed loop — sense, predict, sustain and employ.

01

Data is fragmented

FDR/HUMS telemetry, MIL-STD-1553B avionics buses, paper/e-MMS logs and depot inventory live in separate silos.

NO SINGLE AIRFRAME TRUTH

02

Maintenance is reactive

Fly-to-failure or rigid calendar TBO means components are replaced too late, or pulled too early with life remaining.

CONDITION IS NOT THE TRIGGER

03

AOG creates cannibalization

When a spare's depot lead time exceeds the component's remaining life, crews strip parts from other jets to keep one flying.

ONE SNAG GROUNDS TWO AIRFRAMES

04

Dispatch is health-blind

Sortie allocation rarely considers fatigue index, subsystem health or RUL, so high-stress missions can go to the wrong airframe.

MISSION MISMATCH

→ **Design goal:** predict degradation early enough to position the right spare, inspect the right subsystem and assign the right aircraft — before an AOG event occurs.

02 · SOLUTION AT A GLANCE

Four pillars, one closed maintenance loop.

Every pillar is implemented in the working prototype and exposed through the same FastAPI backend, so the dashboard behaves as one system rather than four disconnected demos.

01

Sense

High-frequency telemetry profile for four flight phases, 21 C-MAPSS sensor channels and derived subsystem health indices.

PHYSICS.PY · /API/TELEMETRY

02

Predict

Three gradient-boosted regressors produce a point RUL, 10th/90th percentile bounds, risk tier and sensor attribution.

MODEL.PY · /API/PREDICT

03

Sustain

IMMOLS-style spares catalog, e-MMS work orders, lead-time-vs-RUL alarms and donor scoring for anti-cannibalization.

LOGISTICS.PY · /API/LOGISTICS

04

Employ

Mission-fit ranking for QRA, CAP, deep strike and ferry profiles using status, RUL, fatigue and subsystem health.

DISPATCH.PY · /API/DISPATCH

Aircraft sensors
EGT, vibration, hydraulics, 1553 bus, G-load

→

Telemetry + health
Physics limits and subsystem health indices

→

RUL + XAI
Point estimate, q10–q90 band, root-cause drivers

→

Twin + depot
3D location, spares lead time, work order

→

Mission dispatch
Ranked, health-aware sortie allocation

↺

✓ **Closed loop:** a predicted RUL below the depot lead time raises an AOG-hazard alert, creates a pre-order, and removes the airframe from mission profiles it can no longer safely fly.

03 · SYSTEM ARCHITECTURE

A browser console over a local Python decision engine.

The frontend is a React single-page application. FastAPI owns the fleet, ML, physics, logistics and dispatch logic, and can serve the built SPA from the same local process.

PRESENTATION

React 19 SPA · TypeScript · Vite · Tailwind CSS v4 · Three.js twin · Recharts · MapLibre GL · Lucide icons

REST / JSON ↕ /API

API

FastAPI · Uvicorn · Pydantic models · OpenAPI /docs · CORS · Static SPA hosting

CALLS ↕ PYTHON ENGINES

INTELLIGENCE

RUL predictor · Physics engine · Risk classifier · XAI attribution · Dispatch scoring · Cannibalization scoring · IETM retrieval

READS ↕ IN-MEMORY STATE

DATA

Fleet registry · Telemetry simulator · C-MAPSS generator · Spares & work orders · IETM knowledge base · Mission profiles

INTERFACES ↕ MODELED

INTERFACES

e-MMS · IMMOLS · 3 / 11 / 7 BRD · MIL-STD-1553B · ARINC-429

Request path

1. React fetches `/api/fleet` on load.
2. Selecting a tail triggers `/api/digital-twin` and `/api/predict`.
3. FastAPI calls the physics and ML engines.
4. JSON returns to the console; Recharts and Three.js render it.
5. A work order posts back to `/api/logistics/order`.

Deployment shape

One local process can serve the API and the compiled SPA. The Windows launcher `start.bat` trains missing models, builds the UI if needed and starts Uvicorn.

! **Demo-state note:** fleet, spares and IETM records are in-memory Python structures. They are intentionally local and reset with the process.

04 · SOFTWARE STACK

What the solution is built with.

Versions below come from `frontend/package.json` and the backend manifests. The stack is deliberately local-first: no cloud ML service, no external LLM and no paid API.

FE

Frontend

React	`19.2.8`	UI runtime
TypeScript	`~6.0.2`	typed contracts
Vite	`8.3`	dev server + build
Tailwind CSS	`4.3.3`	Notion-style UI
Three.js	`0.186.1`	WebGL digital twin
Recharts	`3.8`	RUL + health charts
MapLibre GL	`6.11.2`	base / map layers
Lucide React	`1.49`	icon system

BE

Backend & API

Python	`3.x`	runtime
FastAPI	`API v2.0.0`	REST + OpenAPI
Uvicorn	`requirements`	ASGI server
Pydantic	`FastAPI`	request validation
pandas / NumPy	`requirements`	data + math
scikit-learn	`requirements`	gradient boosting
joblib	`requirements`	model artifacts
pytest / TestClient	`12 tests`	API + engine checks

ML

Models & math

HistGradientBoostingRegressor — point estimate plus two quantile models (10th and 90th percentile). Target is RUL clipped at 125 cycles; features are 3 operational settings + 21 C-MAPSS sensor channels.

24 features · q10 / q90 · 125-cycle clip · NASA PHM score

OP

Delivery & operations

Local/air-gapped deployment, one-command Windows launcher, FastAPI static hosting, CSV export, print-ready console and no external service dependency.

start.bat · uvicorn :8000 · Vite proxy /api · CSV export

! **Not an LLM product:** "Raksha-Mitra" is a deterministic IETM/SOP retrieval engine over a curated fault knowledge base. It does not call OpenAI, Gemini or any cloud model.

05 · MACHINE-LEARNING MODEL

Three gradient-boosted models, one calibrated RUL envelope.

The model predicts Remaining Useful Life in cycles, converts it to flight hours and reports a 10th–90th percentile band so maintainers can plan for the pessimistic case, not just the average.

1 **Data generation**
Deterministic **C-MAPSS FD001-profile** run-to-failure harness: **120 engines**, seed 42, healthy plateau followed by Weibull/exponential wear. Each row has **3 settings + 21 sensors**.

2 **Target definition**
RUL is clipped at **125 cycles** (piecewise upper limit) so the model learns the degradation zone rather than an unbounded healthy plateau.

3 **Engine-level split**
75 / 25 split by engine, not by cycle. This gives **30 held-out engines** and **7,693 test cycles** with no cycle leakage.

4 **Model family**
scikit-learn HistGradientBoostingRegressor: point model (150 iterations, lr 0.08, depth 7) and quantile models q10/q90 (100 iterations, lr 0.08, depth 6).

5 **Inference output**
RUL cycles × **1.4 h/cycle** → flight hours; risk tiers **NOMINAL / ELEVATED / HIGH / AOG**; trajectory and SHAP-style sensor attribution.

9.13
TEST MAE
cycles

0.893
R²
explained variance

13.20
RMSE
cycles

91.4%
BAND COVERAGE
q10–q90

Model artifacts

```
rul_lgbm.joblib
rul_q10.joblib
rul_q90.joblib
benchmark_metrics.json
```

✓ **Graceful fallback:** if artifacts are absent, `model.py` uses a calibrated wear-index surrogate (T50, P30, PS30, BPR) so the demo still returns a prediction.

XAI **Explainability:** the console shows attribution for T50 / `s4`, P30 / `s7`, core vibration and bypass ratio / `s15` — implemented as a deterministic, SHAP-style engineering contribution model, not a SHAP library call.

06 · PHYSICS ENGINE & DIGITAL TWIN

Physics gives the limits. ML learns the wear.

The physics layer turns raw telemetry into interpretable engineering indicators, then the digital twin places those indicators on the airframe so a maintainer can see where the problem is.

EGT margin

`EGT_margin = max(0, 880 − EGT)`

A shrinking margin is the primary physical signal of turbine creep and thermal degradation.

HPC efficiency

`η = (p30/p24)^((γ−1)/γ) / (T30/T24)`

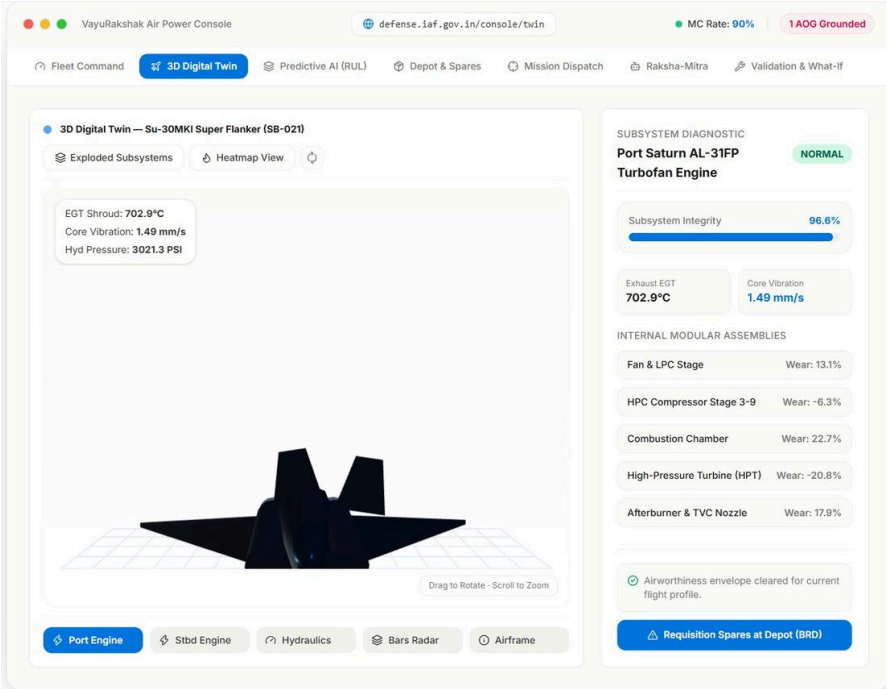
γ = 1.4; drop is measured against a nominal 0.885 efficiency.

Vibration harmonics

Fan and core RMS are tracked with 1X/2X rotor harmonics. Thresholds: 1.5 / 2.8 / 4.5 mm/s for fan; 2.0 / 3.5 / 5.5 mm/s for core.

Fatigue index

`D = Σ(ΔG^3.2 / 1.5e5)` for ΔG > 1.5; simplified rainflow counting with Palmgren–Miner cumulative damage.



Live console: Three.js WebGL twin with exploded subsystems, heatmap mode, subsystem inspector and telemetry readouts.

Subsystem health index

Weighted overall health: **40% propulsion**, **25% hydraulics**, **20% avionics**, **15% airframe**. Five groups are rendered: port/starboard engines, hydraulics, avionics/radar and airframe structure.

✓ **Hybrid rationale:** physics provides fast, interpretable redlines and simulation; the ML model learns non-linear wear patterns from run-to-failure histories. Neither is used alone.

07 · PLATFORM MODULES & DATA LAYER

Eight modules, one operating picture.

The console exposes seven working tabs plus a requisition modal. Every module reads from the same fleet registry, so a prediction, a spare and a sortie decision stay consistent.

01 · Fleet Command

MC/FMC/PMC/AOG counts, base and status filters, critical AOG alerts.

`/API/FLEET`

02 · 3D Digital Twin

Exploded subsystems, thermal/stress heatmap, live readouts and component wear.

`/API/DIGITAL-TWIN/{TAIL}`

03 · Predictive AI

Point RUL, q10/q90 band, degradation trajectory and sensor attribution.

`/API/PREDICT/{TAIL}`

04 · Depot & Spares

IMMOLS-style catalog, e-MMS work orders and lead-time-vs-RUL deficit alerts.

`/API/LOGISTICS`

05 · Anti-Cannibalization

Donor feasibility score, swap time vs depot replenishment, fleet-impact assessment.

`/API/LOGISTICS/CANNIBALIZATION`

06 · Mission Dispatch

QRA, CAP, deep strike and ferry profiles; eligibility gates and ranked allocation.

`/API/DISPATCH/RECOMMEND`

07 · Raksha-Mitra

Four fault families with SOP steps, required tools and estimated turnaround.

`/API/ASSISTANT/QUERY`

08 · Validation & What-If

C-MAPSS metrics, combat/dust/high-G stress simulation and fleet CSV export.

`/API/BENCHMARK · /API/WHATIF`

Fleet & bases

10 aircraft across **4 types** — Su-30MKI, Tejas Mk1A, Rafale, MiG-29UPG — at **6 airbases**: Ambala, Halwara, Sulur, Tezpur, Bareilly and Pune.

Depots & spares

3 BRDs — 3 BRD Chandigarh, 11 BRD Ojhar/Nashik, 7 BRD Tughlakabad. **5 NSN-coded items** with 12–36 h lead times and live stock levels.

Knowledge & missions

4 IETM fault families (EGT, vibration, hydraulic, 1553 bus) and **4 mission profiles** with explicit G, RUL, fatigue and health gates.

! **Integration stance:** data contracts are modeled on IAF e-MMS/IMMOLS and BRD workflows, but the current build uses simulated records. It is a working prototype, not a live depot connection.

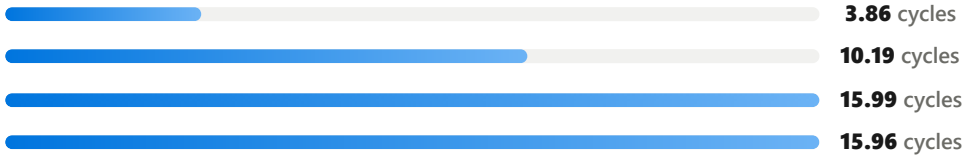
08 · VALIDATION

Measured on held-out engines, not on the training set.

The benchmark script performs an engine-level split, evaluates the point and quantile models, and writes the report consumed by the validation console.

9.13	13.20	0.893	91.4%	22.7k
TEST MAE	RMSE	R^2	Q10–Q90 COVERAGE	NASA PHM SCORE
cycles	cycles	explained variance	target 80%	22,692.15

MAE by remaining-life window



Values from `benchmark_metrics.json`; engine-level hold-out with 30 test engines and 7,693 test cycles. Lower is better.

What the numbers mean

3.86 cycles MAE in the 0–25 window — the highest precision exactly where flight safety matters. **91.4%** of true RUL values fall inside the q10–q90 band, above the 80% target.

Benchmark scope

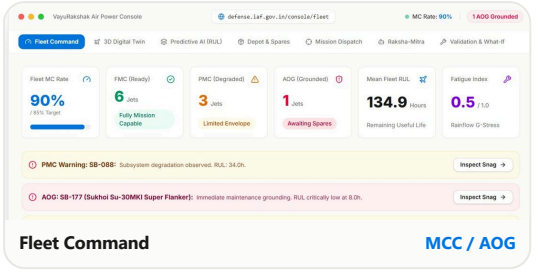
The shipped build uses a deterministic **120-engine C-MAPSS-profile generator** so the demo runs offline. It is not yet trained on recorded FD001 files or live squadron telemetry. Metrics are reproducible with `python -m app.train`.

! **Claim boundary:** these are benchmark results on a controlled run-to-failure dataset, not a flight-qualified certification. Field calibration and recorded-data validation are the next step.

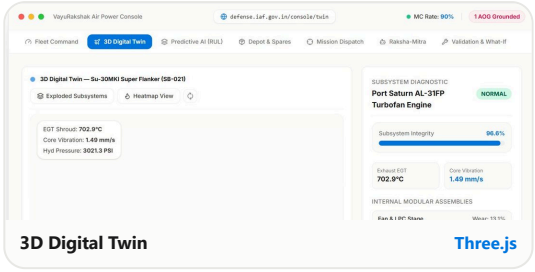
09 · WORKING PROTOTYPE

The console is live, not a mock-up.

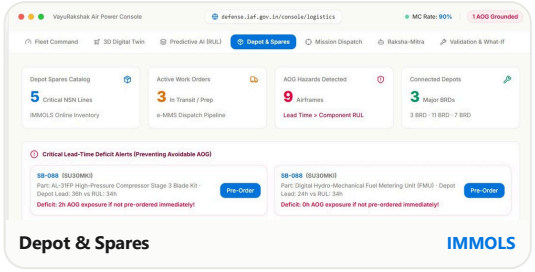
Captured from the running React console at 1440px wide, backed by the FastAPI service. Each tab is interactive; the screenshots below are the actual rendered UI.



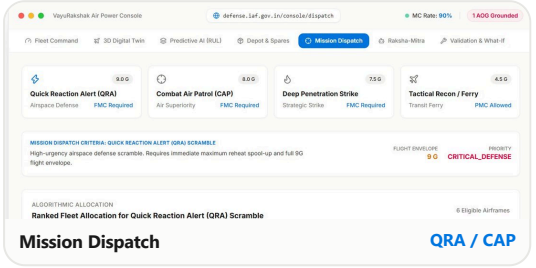
Fleet Command — MCC / AOG



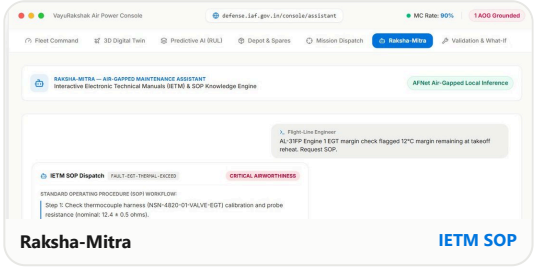
3D Digital Twin — Three.js



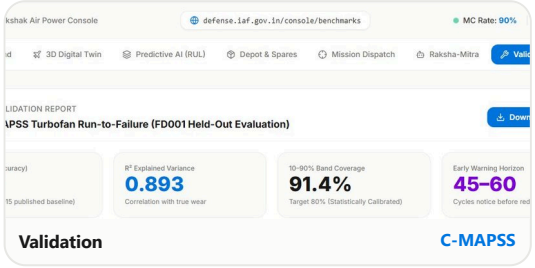
Depot & Spares — IMMOLS



Mission Dispatch — QRA / CAP



Raksha-Mitra — IETM SOP



Validation — C-MAPSS

→ **Interaction path:** select an airframe in Fleet Command → inspect its 3D twin and RUL → pre-order the flagged spare → dispatch or reserve the airframe based on mission fit.

10 · DEPLOYMENT, SECURITY & ROADMAP

Built to run inside the station, not in the cloud.

The current prototype runs entirely locally. The roadmap is about connecting it to real maintenance systems — not replacing the model with a hosted service.

Deployment posture

Local/air-gapped execution; FastAPI serves the API and the built SPA from one process. `start.bat` trains missing models, builds the UI if needed and launches Uvicorn. No cloud ML, no external LLM, no paid API.

Security & data notes

Demo data is in-memory and resets with the process. CORS is open for local development; production requires restricted origins, authentication/RBAC, audit logging and a database-backed store. These are deliberate next-step items, not hidden claims.

Verified in the build

12 backend tests cover fleet, dossier, telemetry, twin, quantile ordering, logistics, work orders, cannibalization, dispatch, assistant, what-if and CSV export. The frontend builds with Vite.

PHASE 1 · 0–3 MONTHS

Integrate

e-MMS/IMMOLS sandbox, ground-test rig feed, real NASA FD001 file loader, field sensor calibration.

PHASE 2 · 3–6 MONTHS

Field trial

Selected Su-30MKI / Tejas squadron, maintainer feedback loop, threshold tuning, reliability tracking.

PHASE 3 · 6–12 MONTHS

Depot rollout

3 BRD / 11 BRD / 7 BRD deployment, RBAC and audit, AFNet hardening, full spares workflow.

! **Status:** working prototype, not flight-certified. The ML, physics, logistics and dispatch engines are functional; live sensor and depot integrations remain roadmap items.

11 · ONE-SLIDE SUMMARY

What was built, with what, and what it proves.

A presentation-ready summary you can read in 60 seconds — or lift directly into a closing slide.

BUILT WITH

- **React 19 + TypeScript + Vite** single-page console
- **Tailwind CSS v4** for the Notion-style interface
- **Three.js** for the 3D digital twin
- **Recharts** for RUL and health charts
- **FastAPI + Uvicorn** for the REST decision engine
- **NumPy, pandas, scikit-learn, joblib** for data and models

INTELLIGENCE

- **HistGradientBoosting** point + q10/q90 RUL models
- **24 features** — 3 settings + 21 C-MAPSS sensors
- **Physics engine** — EGT margin, HPC efficiency, vibration, fatigue
- **SHAP-style XAI** for root-cause sensor attribution
- **Lead-time-vs-RUL** spares alerts and donor scoring
- **Mission-fit dispatch** across four operational profiles

PROOF

- **9.13 cycles** test MAE on 30 held-out engines
- **13.20 cycles** RMSE · **0.893** R^2
- **91.4%** q10–q90 band coverage
- **3.86 cycles** MAE in the 0–25 cycle window
- **7,693** test cycles · engine-level split
- **12** backend tests · working 7-tab console

From reactive maintenance to predictive fleet availability — sovereign, explainable and air-gapped.

Run: **start.bat** → http://127.0.0.1:8000 · Dossier: **/solution.html** · SIH 2026 · Problem Statement 26249 · Ministry of Defence (DSSC)